

[CO₂-AT₁]

computer architecture

Shaikh. Babii

192525321

CSA1208

1. Signal 8-bit addition and subtraction.

Given

W.K.T

$$\begin{array}{r} 2 \overline{) 45} \\ 2 \overline{) 22} - 1 \\ 2 \overline{) 11} - 0 \\ 2 \overline{) 5} - 1 \\ 2 \overline{) 2} - 1 \\ 1 - 0 \end{array}$$

$$\begin{array}{r} 2 \overline{) 23} \\ 2 \overline{) 11} - 1 \\ 2 \overline{) 5} - 1 \\ 2 \overline{) 2} - 0 \\ 1 - 0 \end{array}$$

$$45 = 101101 \\ 00101101$$

$$23 = 1011 \\ 00010111$$

-23

$$\begin{array}{r} 00010111 \\ 11101000 \\ \hline 11101001 \end{array}$$

addition

$$\begin{array}{r} 00101101 \\ 11101001 \\ \hline 00010110 \\ = +22 \end{array}$$

subtraction

$$45 - (-23) = 45 + 23 = 68$$

$$\begin{array}{r} 00101101 \\ 00010111 \\ \hline 01000100 \\ = +68 \end{array}$$

2. 4 bit carry load ahead adder.

A carry load ahead adder (CLA) is used to reduce the delay called by carry propagation in a pipeline carry adder.

1. generate and propagate signals:

$$g_i = A_i B_i \quad \text{propagate signal} \quad p_i = A_i \oplus B_i$$

2. Carry equations

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 c_0 + p_1 p_0 c_0$$

$$c_3 = g_2 + p_2 c_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$

$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 g_0$$

$$s_i = p_i \oplus c_i$$

3. working of CLA:

The CLA calculates for carry signals directly using the generate and propagate signals. The carry does not need to wait for the previous full adder stage. c_1, c_2, c_3, c_4 .

4. comparison

carry calculation

speed

propagated delay

Hardware complexity

performance.

sequential

slow

high

low

lower

parallel

low

low

higher

higher

3.

Booth's multiplication algorithm.

sp

multiples

$$(-6) + (+5)$$

Given

$$M = -6$$

$$Q = +5$$

$$-6 \times 5 = -30$$

step 1: Represent -6

$$+6 = 00110$$

$$\text{its complement} = \begin{array}{r} 1101 \\ +1 \\ \hline 11010 \end{array}$$

$$M = -6 = 11010$$

$$A = 00000$$

$$Q = 00101$$

$$M = 11010$$

step 2: Represent +5

$$\begin{array}{r} 2/5 \\ 2/2-1 \\ \hline 1-0 \end{array}$$

$$Q = 00101$$

Booth's algorithm rules:-Q₀ Q₋₁

0 0

operation

no operation

0 1

$$A = A + M$$

1 0

$$A = A - M$$

1 1

no operation

after every operation an arithmetic right shift is performed.

Booth's algorithm table:

step	A	Q	Q-1	Q0Q-1	
initial	00000	00101	0	-	-
1	00110	00101	0	10	A-m
shift	00011	00010	1	-	Right shift
2	11101	00010	1	01	A+m
shift	11110	10001	0	-	R.S
3	00100	10001	1	10	A-m
shift	00010	01000	1	-	R.S
4	11100	01000	0	01	A+m
shift	11110	00100	0	-	R.S
5	11110	00100	0	00	no operation
shift	11111	00010	-	-	R.S.

$$AQ = 111100010$$

$$Q_{15} = 1111100010_2 = -30_{10}$$

$$(-6) \times (5) = -30$$

advantages:

- It directly handles signed numbers.
- It uses 2^15 complement representation.
- It reduces unnecessary addition operation.
- Right shift procedure for sign
- It is efficient for signed multiplication.

4. restoring and non restoring division.

$$13 \div 3$$

$$\begin{array}{r} 2 \overline{) 13} \\ 2 \overline{) 6} - 6 \\ 2 \overline{) 3} - 0 \\ \underline{1} - 1 \end{array}$$

$$= (1101)_2$$

$$2 \overline{) 3}$$

$$(0011)_2$$

restoring division

$$A = 0000$$

$$Q = 1101$$

$$M = 0001$$

calculation:-

step

operation

Result

1

shift & subtraction

-

2.

shift & subtraction

negative $\rightarrow$ Restore

shift & subtraction

positive $\rightarrow Q_0 = 1$

3.

negative $\rightarrow$ positive

4.

shift and subtraction negative $\rightarrow$ Restore

$$A = 00001$$

$$Q = 0100$$

$$Q = 0100_2$$

$$P = 0001_2$$

3, non Restoring division.

In non-restoring division, a negative partial is not immediately Restored.

5) IEEE 754 Representation of -13.25.

step 1: binary conversion

$$13 = 1101_2$$

$$0.25 = 0.01_2$$

$$-13.25 = -1101.01_2$$

$$= 1101.01 = 110101 \times 2^3$$

$$S = 1, E = 3, F = 10101$$

Sign precision:

Fraction :-

$$1010100000000000$$

$$1100000101010000000000$$

therefore

$$110000010101010000000000$$

Hexadecimal

$$C1640000.$$